Author Response to Reviews of

DAENet: Making Strong Anonymity Scale in a Fully Decentralized Network

Tianxiang Shen, Jianyu Jiang, Yunpeng Jiang, Xusheng Chen, Ji Qi, Shixiong Zhao, Fengwei Zhang, Xiapu Luo, Heming Cui

IEEE Transactions on Dependable and Secure Computing,
`Manuscript ID:TDSC-2019-11-0652`

Dear Referees,

Thank you very much for your valuable comments. We are profoundly grateful for the feedback, which greatly helped us improve the quality of the origin manuscript. Accordingly, we have modified the manuscript by taking into consideration both reviewers' requests. We take it as a positive gesture that both referees have submitted positive remarks about the manuscript. The revised manuscript has been re-written by taking into account all these recommendations. The changes in the revised manuscript are catalogued here. We have, thus, replied to the Reviewers' comments, and indicated clearly what we have changed.

RC:*Reviewer Comment*, AR:*Author Response*, □ Manuscript text

1 Associate Editor

RC: Please carefully address every comment from each reviewer.

AR: Thank you for allowing us the chance to improve our work through the revised manuscript.We have tried to include all the suggested changes in the updated manuscript. Individual reviewer-specific comments, which have been incorporated in the updated manuscript, are specifically catalogued in this document.

2 Reviewer #1

RC: I like the idea of the paper, but I just don't think it's publishable in the current format. The writing, organization, and evaluation need to be significantly improved. It looks like you used the Loopix paper as a guideline for your evaluation, but that's a very different network, and so your eval is going to be different.

AR: Thank you for your like of our ideas and pointing out the shortcomings of our original manuscript. For writing, we have corrected all mistakes, and given the revised work a thorough read-through to reduce writing errors further. For paper organization, we have re-written the main part of this paper to highlight the contribution of this work better and avoid repetition and unnecessary section pointers. For evaluation, we have re-written the evaluation setup to clearly introduce our motivation of the usage of baselines systems such as Loopix.

RC: The discussion of Tor vulnerabilities in the introduction is dated (based on a paper from 2005), and does not take into account more recent Tor improvements,

such as cover traffic, guard nodes, and etc.

AR: We have moved the detailed description of Tor to the related work, and discussed Tor's state-of-art vulnerabilities and defense techniques (e.g., cover traffic and guard node selection). We have included the following information in Section 8 of the revised manuscript.

> The most popular relay-based anonymous communication system is Tor [15]. Due to its popularity and transparent development processes [61], many researchers have explored attacks that can de-anonymize Tor users and hidden-service providers by monitoring the network traffic. Recent attack directions for Tor include website fingerprinting [62], [63], [64], [65], traffic correlation [10], [11], [12], [66], congestion attack [67], [68] and targeted DoS [39], [69], [70]. Meanwhile, researchers also propose methods to enhance Tor's security by optimizing the bandwidth report for selecting guard nodes [71] and monitoring circuit construction [72]. Also, some recent Tor improvements consider generating cover traffic within middle routers of circuits, such that the middle routers can hide any relationship between compromised entry and exit nodes [73], [74].

RC: How/who assigns ID's? (section 2.2) You discuss this later, but not here.

AR: We have given more details of DAENet's underlying communication network in the revised manuscript. Specifically, in the underlying network, each participant joins the network by sending a *join* request to a known DAENet node. The node will assign an identifier to the participant by using the consistent hashing technique and help the participant set up its routing table by several rounds of consultations to its neighbors. In DAENet, the node is called the *guarder node*. The guarder node also serves as an attestation server to verify whether an unmodified DAENet's program is executed inside a real SGX host.

RC: DAE v.s. SGX-Tor – I think this part is in the wrong place. The comparison vs SGX-Tor makes a lot of claims that haven't been supported or even explained. Possibly move this later in the paper.

AR: Thank you for pointing out the organization problem. We have moved this comparison to the related work.

RC: I don't think it's so easy to just say "SGX is trusted" when there are several known vulnerabilities that compromise the security of the system. For SGX-Tor, if SGX fails, you're left with Tor, which is somewhat secure. With your system, if SGX is compromised, there is no security left at all. This is a major difference, and I think it merits at least a discussion of the impact of known SGX flaws on your system.

AR: Thank you for your valuable insights. We have added the following paragraph to discuss the SGX vulnerabilities that could happen and how to defend against them in section 4.1.

We notice that an SGX may be compromised because of SGX vulnerabilities [48], further compromising the anonymity provided by DAENet. DAENet solves this problem using two approaches. First, such vulnerabilities can usually be fixed through CPU microcode updates [49], and such updates increase the Security Version Number (SVN) used for attestations. DAENet's guarder node checks the latest SVN within the network and rejects nodes with SVN less than this value during attestations, so nodes with out-of-date microcode (i.e., contain potentially compromised SGX) cannot join the network. Second, for vulnerabilities that cannot be fixed through CPU microcode updates, Intel returns a revocation certificate list during attestations. DAENet rejects attestation reports signed by these certificates and avoids the admissions of nodes with SGX vulnerabilities that cannot be fixed.

RC: You discuss the different attacks you will protect against, but you don't specify *how* the system will protect against them. Contrast this with Section 3.3 of the SGX-Tor paper to see what I'm talking about.

AR: Thank you for your comments. In the third chapter where we talked about the security goals before, we have added corresponding defense techniques for clearer elaboration. Specifically, there are three security goals and corresponding privacy approaches. First, to defend against global passive attacks, we shuffle real messages with indistinguishable dummy messages, and disseminate messages to each participant's neighbors with full randomness to mess up the orders of input and output packets. Second, to defend against eclipse attacks, we use close-loop packet to detect malicious packet drops by verifying each on-path relay's *Proof-of-Forwarding* generated inside SGX. Last, to defend against traceback attacks, we use randomly generated dead drop nodes to build up different and unpredictable routing circuits in each communication round, preventing active attackers tracking and revealing sender identities.

RC: Organization – the organization of the paper makes it difficult to read. For example, you often refer to things you haven't discussed yet by saying "see section XXX". However, often this is critical information to understanding the thing you are currently discussing. Skipping around in the paper is hard on the reader, and makes it difficult to evaluate what you've done. This suggests that you may need to reorganize your paper to more logically present your work. There is also a large amount of repetition in the paper. For example in 3.2 line 54 you talk about dead drop nodes, then 3.3 you discuss them again, and repeat most of 3.2 and then add a bit to the discussion.

For the organization, we have completely re-organized the flow and the structure of the paper. We have re-written the introduction to help readers understand the requirements of practical anonymous communication systems and the position of DAENet, re-written the background to give more details of anonymous communication network and the underlying P2P structure, and re-written the overview to correlate every attack and defense techniques. In addition, all chapters are updated with a thorough read-through to avoid mistakes such as undefined terms, misleading section pointers and etc.

RC: The theorems in this section aren't actually theorems. They're just statements of equations that come directly from your formulations. Also, you refer to Theorem 2 as a "lemma" in the text preceding it, which is not a theorem. You also say it's an "upper bound" but there is no inequality, which an upper bound would imply. You

later state that 1/y is the is "the upper bound for an adversary to correctly link the input message x and the corresponding output message x1." which might be true, but this is not derived or demonstrated in your paper. This looks like what they did in the Loopix paper, but they provided proofs in an appendix, which you do not.

AR: Thank you for pointing out the writing mistake in this paper. The previous Theorem 1 that described the process of pulling messages from shuffle pools has been refactored to be a statement in the revised manuscript. The statement is included in section 4.2:

> Denote $\mathcal{N}$ as the total number of participants in the network and k as the number of empty shuffle pools of Alice's all successors. Derived from previous statements, Alice pulls messages from $\log\mathcal{N} - k$ shuffle pools in each protocol run. The pulling process takes the form of Binomial distribution $\mathcal{X} \sim B(\log\mathcal{N}, \alpha)$ where the discrete probability α is the *expected shuffle rate*. In each round, the expected total number of real application messages and dummy messages for Alice to send is $\alpha(\log\mathcal{N} - k)$ and $k + (1 - \alpha)(\log\mathcal{N} - k)$, respectively.

The previous Theorem 2 has been refactored to a Claim which describes the probability of correctly linking an arriving message with an outgoing message in a host. We give an upper bound probability 1/y because the outgoing message is from the host's shuffle pool, hence limited to the total number of existing messages in current host. As there are totally y messages as we defined, the upper bound on the probability that adversaries correctly do the traffic correlation is thus 1/y. This inference applies to other shuffled-based systems that defends against traffic correlation attacks as well. Loopix handles traffic correlation attack and also gives an upper bound on the attack probability (i.e., 1/(l+k)) because Loopix assumes a shuffle pool is mixed l+k messages presently.

RC: The malicious host detection only seems to work if *all* network packets are dropped. What if the host is only dropping a fraction of messages? It's also unclear how you form the path such that you know who all the participants in the loop path will be, given that you may not be connected to them?

AR: There are two cases of malicious packet drops in DAENet. First, if active attackers drop all messages of a host, we guarantee that the loop message detection protocol can detect it. Second, if active attackers selectively drop messages, our detection protocol can resist with probabilistic defense according to τ, which is an *expected detection probability* chosen by the participant. We have added the discussion of the guarantee of our loop message detection protocol in **section 4.3**, and proved the probabilistic defense in **Appendix A** of our revised manuscript.
Regarding the relays on the loop path, they are deterministically selected with reference to a random bit string b_s generated within enclaves: each relay on the path selects its b_sth neighbor as the next hop. Thus, a participant can request for all the on-path relays by asking its neighbors iteratively.

RC: I don't think I've ever seen the word phishing used in this context.

AR: We have changed the name of the attack on revealing sender identities in the revised manuscript from "phishing attack" to "traceback attack", in order to avoid misunderstandings of the original phishing attacks used in other security domains (e.g., web browsing). In section 4.4, we have added more description of the traceback attacks.

RC: One malicious dead drop node won't compromise sender/receiver confidentiality, but what about multiple nodes. Specifically, what fraction of the dead drop nodes

need to be non-malicious? This is a key number because it's the easiest place to compromise the network.

AR: Thank you for your valuable comments. If one dead drop node is malicious, the only thing it can do is refuse to exchange the payload and forward packets. If so, the participant can exchange the payload by sending to another unused dead drop nodes in next communication round without compromising anonymity. A malicious dead drop node cannot correlate sender/receiver because the input and output packet order is totally mixed by shuffles. This guarantee still holds even if multiple dead drop nodes are malicious because (1) message shuffles totally mess up the orders of messages, even multiple malicious dead drop nodes cannot link an input and output message and further re-construct a routing circuit (2) different dead drop nodes serve on different routing circuits, thus malicious dead drop nodes cannot make consensus on the real message senders.

RC: epsilon for your network is quite high, even for large numbers of nodes. (Compare this with Loopix.) You also only analyze this for two parameters of your network, and it seems like you combine shuffle rate and emission rate? Shouldn't you explore each parameter independently?

AR: Compare DAENet's likelihood difference with Loopix, in the worst case, DAENet's likelihood is smaller, while in the best case Loopix's likelihood is smaller than DAENet's. However, Loopix achieves this by sacrificing communication latency with increasing delays. We have added the likelihood comparison with Loopix in the revised manuscript.

Loopix also uses likelihood to evaluate its defending capability against global traffic attacks. Even if Loopix's likelihood can be smaller than DAENet, it incurs additional delay in each mix node. In Loopix's likelihood evaluation setup (i.e., a topology of 3 layers with 3 mix nodes per layer), when Loopix achieves comparable likelihood as DAENet (0.25), it incurs additional 1s delay in each mix node. Thus, Loopix sacrifices 3s latency throughout all three layers of shuffles which is larger than DAENet's end-to-end communication latency (see §6).

We analyze the likelihood from two aspects: the scale of the network and the shuffle rate in each host. We do not study the likelihood difference with different message emission rate because DAENet enforces participants to have the same emission rate which can prevent adversaries from learning sensitive information by observing either a participant is idle or busy. Thus, changing the emission rate will not affect anonymity.

RC: 6.1 Line 51-52 – I'm not sure what you mean when you say "DAENet can identifies them". There is no explanation here for the lack in increase in latency. This also raises the question of what happens with anonymity when almost every packet is real and not cover traffic.

RC: Bandwidth usage – I don't see how you avoid network bandwidth for dummy messages. It says here they replace dummy messages with control messages, but how many control messages are there? And if you limit dummy traffic, don't you either limit throughput or limit the anonymity provided by cover traffic?

AR: We have updated the evaluation chapter to give more concrete explanations. In regard to anonymity, since real traffic and dummy traffic are equally shuffled, when almost every packet is

real and not cover traffic, the anonymity guarantee will not be affected: for global passive attackers, they still cannot correlate an input and output message because each participant shuffles messages that messes up the order of messages, and all messages are mixed regardless of whether a message is real or not. For active attackers that drop messages, they would have a larger probability of dropping a real message. However, the drop of real messages can still be detected by the detection protocol.

3 Reviewer #2

RC: The presentation of this paper is satisfactory. Nonetheless, the following issues should be addressed for better readability:

AR: Thank you for your encouraging comments.

RC: Can you guarantee that all malicious drops are detected? How periodic is the detection performed? Can you guarantee the security within a time interval when the misbehaves are not detected? Is there any tradeoff for choosing the periodicity?

AR: There are two cases of malicious packet drops in DAENet. First, if active attackers drop all messages of a host, we guarantee that the loop message detection protocol can detect it. Second, if active attackers selectively drop messages, our detection protocol can resist with probabilistic defense according to τ, which is an *expected detection probability* chosen by the participant. We have added the discussion of the guarantee of our loop message detection protocol in **section 4.3**, and proved the probabilistic defense in **Appendix A** of our revised manuscript.

RC: It is questionable that the use of SGX provides enhanced security as it relies on an SGX assumption (i.e., it is not subject to side channel attacks) and periodic detection of active attacks. Is it better to emphasize efficiency and scalability (even robustness)?

AR: Thank you for your valuable insights. For the SGX assumption, we have added a discussion of side-channel attacks in section 9. Also, we have added a discussion about the effect of SGX vulnerabilities on DAENet and corresponding defense at the end of section 4.1:

> We notice that an SGX may be compromised because of SGX vulnerabilities [48], further compromising the anonymity provided by DAENet. DAENet solves this problem using two approaches. First, such vulnerabilities can usually be fixed through CPU microcode updates [49], and such updates increase the Security Version Number (SVN) used for attestations. DAENet's guarder node checks the latest SVN within the network and rejects nodes with SVN less than this value during attestations, so nodes with out-of-date microcode (i.e., contain potentially compromised SGX) cannot join the network. Second, for vulnerabilities that cannot be fixed through CPU microcode updates, Intel returns a revocation certificate list during attestations. DAENet rejects attestation reports signed by these certificates and avoids the admissions of nodes with SGX vulnerabilities that cannot be fixed.

To better emphasize our strengths, we have highlighted the importance of low-latency as the first requirement of practical anonymous communication systems from the beginning of the revised manuscript, and shown the efficiency, scalability and robustness of our system.

RC: At the end of section 2.2, please clarify that N is the total number of participants.

RC: In section 3.1.1, it says, "DAENet allows arbitrary number of compromised participants in the network, but requires an adequate number of honest participants to ensure message deliveries." Here "arbitrary number" is contradictory to "adequate number". Please revise.

RC: It says, "DAENet requires at least O(logN) out of N honest participants". It

should be revised to "DAENet requires at least O(logN) honest participants out of total N participants."

AR: Thank you for pointing out these writing mistakes in our paper. We have incorporated all your suggestions in the revised manuscript.

RC: At the end of section 4.1, it says "a malicious joiner node will fail to pass the attestation". Please clarify why.

AR: We have added the following explanation at the end of section 4.1:

> Since SGX remote attestation can help verify the integrity of the running SGX code, if a malicious *guarder node* refuses to admit benign nodes or tries to admit specific participants, the malicious *guarder node*'s code integrity is broken. Hence the malicious *guarder node* will fail to pass the attestation.

RC: For theorem 2, what is the relationship between y and t,k in extreme cases? Does this theorem hold in extreme cases even if Alice is compromised?

AR: The value of y, t and k is independent. If Alice receives an input message, the message's next hop has equal probability of being any of Alice's neighbors. Thus, it is likely for Alice to hold none messages in some of its shuffle pools. In other word, whatever the value of y is, k - the number of empty shuffle pools is independent of y and can be any non-negative value while k $<\log \mathcal{N}$ still holds (N is the total number of participants in the network). This can be applied to t as well. If Alice is compromised, Alice can arbitrarily drop messages but this theorem still holds because Alice cannot intervene the shuffling process within SGX and correlate pairs of input and output messages.

RC: In section 4.3, a random bit bs is specified by Sender S. Is bs a random bit or a random bit string? If it is the former, then either the 0-th neighbor or the 1-th neighbor is selected at each hop. Please double check.

AR: Thank you for pointing it out. It should be a random bit string that can be mapped to one of the neighbors. We have modified it and given the revised work a thorough read-through to reduce writing errors further.

RC: In section 4.5, is the existence of service providers and brokers contradictory to the P2P feature of DAENet and its security against DoS attack?

AR: The existence of service providers and brokers is not contradictory because the roles are ephemeral and stealthy. The role is ephemeral because Alice can assign different broker nodes to serve it's registration requests, and a broker node will terminate its job of handling Alice's requests immediately after Alice chooses other broker nodes. The role is stealthy because the traffic pattern of a broker node is the same as other nodes in the network, hence global attackers cannot observe a special broker node.

With gratitude,
Tianxiang Shen
Author of the manuscript